# INTERVIEW PREP MASTER PLAN

**AI Engineer · Backend Engineer · Full-Stack Developer**

FastAPI · SQLModel · PostgreSQL · Redis · Celery · LangChain · LangGraph · Docker · Next.js

## PREPARATION TIMELINE OVERVIEW

| Day(s) | Stack | Focus | Role Relevance | Time |
|---|---|---|---|---|
| 1 | **FastAPI Core** | Params, Headers, Cookies, File Upload, DI, CORS, Error Handling | BE + FS | 9 hrs |
| 2 | **FastAPI Advanced** | JWT, Middleware, WebSockets, Background Tasks, SSE Streaming, Testing | BE + FS | 9 hrs |
| 3 | **SQLModel + PostgreSQL** | Models, CRUD, Async Sessions, Alembic, JSONB, Indexes, FTS, Pooling | BE + FS | 9 hrs |
| 4 | **Redis** | Data Structures, Caching, Pub/Sub, Rate Limiting, Locks, FastAPI integration | BE + AI | 9 hrs |
| 5 | **Celery** | Tasks, Workers, Beat, Retry, Chain/Group, Flower, FastAPI Integration | BE | 9 hrs |
| 6 | **LangChain + LangGraph** | LCEL, Tools, RAG, Memory, StateGraph, HITL, Multi-Agent | AI | 9 hrs |
| 7 | **Docker** | Dockerfile, Multi-stage, Compose, Networks, Volumes, Health Checks | BE + AI + FS | 9 hrs |
| 8 | **Next.js** | App Router, Server/Client Components, API Routes, Data Fetching, Auth | FS | 9 hrs |
| All | **DSA** | 2 hrs/day — Arrays, Trees, Graphs, DP, Greedy, Sliding Window etc. | All Roles | 2 hrs/day |
| All | **System Design** | 45 min/day — Core concepts integrated into each stack day | All Roles | 45 min/day |

## ● DAILY SCHEDULE STRUCTURE

*Follow this time structure every single day. DSA in the morning keeps your mind sharp before the main stack work.*

| Time Block | Activity | Duration | Notes |
|---|---|---|---|
| **6:30 – 7:00 AM** | Wake up + DSA warm-up (review previous) | **30 min** | *Read yesterday's solution* |
| **7:00 – 8:30 AM** | DSA — LeetCode problems (main session) | **1.5 hrs** | *2-3 problems: 1 easy, 1-2 medium* |
| **8:30 – 9:00 AM** | Break + System Design reading (15 min) | **30 min** | *Pick 1 concept from the SD list* |
| **9:00 AM – 1:00 PM** | Main Stack — Concepts + Official Docs | **4 hrs** | *Read → Understand → Test each topic* |
| **1:00 – 2:00 PM** | Lunch + rest | **1 hr** | *Step away from screen* |
| **2:00 – 5:00 PM** | Main Stack — Practical building | **3 hrs** | *Build endpoints, test in Postman/curl* |
| **5:00 – 5:45 PM** | System Design — deep dive | **45 min** | *Think through: how would I design X?* |
| **5:45 – 6:30 PM** | Revision + personal notes | **45 min** | *Write what you learned in your own words* |

| | |
|---|---|
| **⚠ DSA Rule** | If a problem takes more than 25 min and you're completely stuck → look at the hint only, not the solution. Move on. Don't let 1 problem eat your whole session. |

| | |
|---|---|
| **📖 Docs Rule** | For every topic: open the official docs FIRST. Read the full section. THEN test it practically by building a small endpoint or script. Don't copy-paste — type it out. |

# DAY 1
FastAPI

## FastAPI — Core Fundamentals

*Master the building blocks that 90% of interviews test*

📖 DOCS | Official Docs: https://fastapi.tiangolo.com/tutorial/

## TOPICS TO COVER

### 01 Path Parameters & Query Parameters

*Foundation of every REST API — interviewers test this to check Pydantic understanding*

→ Path params with type hints — {item_id: int}, {name: str}
→ Query params with defaults and Optional — q: Optional[str] = None
→ Enum path parameters for predefined choices
→ Path() and Query() with validation: min_length, max_value, regex
→ Multiple query params, required vs optional

### 02 Request Body & Pydantic Models

*This is where FastAPI's power comes from — understand Pydantic deeply*

→ BaseModel — define request schema, automatic validation
→ Nested models (model inside model)
→ Body() for explicit single/multiple body params
→ Field() for validation — ge, le, min_length, description
→ model_validator and field_validator (Pydantic v2)
→ Extra fields config: model_config = ConfigDict(extra='forbid')

### 03 Response Models & Status Codes

*Critical for not leaking sensitive fields (e.g. password hash) — common interview pitfall*

→ response_model= to control what gets serialized back
→ response_model_exclude_unset, response_model_exclude
→ status_code= — 200, 201, 204, custom
→ JSONResponse, Response, PlainTextResponse
→ Union response types for different return shapes

### 04 Headers — Request & Response

*Very practical — every production API uses custom headers for auth tokens, tracing, versioning*

→ Reading request headers: Header() — auto underscore-to-hyphen conversion
→ Setting response headers via Response object or custom Response
→ Custom headers in middleware
→ X-Request-ID pattern — tracing headers
→ Duplicate headers (list type)

### 05 Cookies — Read & Set

*Session-based auth and CSRF protection — know the security attributes cold*

→ Reading cookies: Cookie() parameter
→ Setting cookies: response.set_cookie(key, value, httponly=True, secure=True)
→ Delete/expire cookies: response.delete_cookie()
→ SameSite, secure, httponly flags — why they matter for security
→ Dependency that extracts session from cookie

## 06 File Uploads

*Common in AI apps (uploading docs for RAG) — know async file reading pattern*

→ Single file upload: File(), UploadFile — read(), chunks
→ Multiple files: List[UploadFile]
→ Form data + file combined: Form() + File()
→ File size validation — manual check on upload
→ Streaming large file reads to avoid memory issues
→ Background file processing after response

## 07 Dependency Injection — Basics

*THE most important FastAPI concept for senior roles — understand the DAG model*

→ Depends() — how it works under the hood (DAG resolution)
→ Dependencies as functions vs classes
→ Shared dependencies across routes
→ Dependencies with yield (setup/teardown — DB sessions)
→ Sub-dependencies (deps that depend on other deps)

## 08 CORS & Middleware Setup

*Every full-stack app needs CORS configured — interviewers ask about security implications*

→ CORSMiddleware — allow_origins, allow_methods, allow_headers, allow_credentials
→ Why '*' in allow_origins breaks credentials
→ Custom middleware with BaseHTTPMiddleware — request/response interception
→ Request timing middleware — add X-Process-Time header
→ Middleware order matters — LIFO stack

## 09 Exception Handling

*Production APIs have uniform error responses — know how to build this from scratch*

→ HTTPException with status_code and detail
→ Custom exception classes
→ Exception handlers: @app.exception_handler(MyException)
→ RequestValidationError override — custom 422 response format
→ Global error response schema for consistent API errors

## 10 Lifespan Events & App Structure

*Interviewers ask: how do you load a model once without reloading on each request? Lifespan.*

→ Lifespan context manager (modern approach vs on_event deprecated)
→ Startup: initialize DB pools, load ML models, warm cache
→ Shutdown: close connections gracefully
→ APIRouter — organizing routes into modules

→ include_router() with prefix and tags

## 🛠️ PRACTICAL EXERCISES

1. Build /users/{user_id} endpoint — path param with int validation, return mock user data
2. Build /search?q=...&limit=10&offset=0 endpoint — query params with validation
3. Create a UserCreate (request) and UserResponse (response) Pydantic model — exclude password from response
4. Build /upload endpoint — accept a PDF file, return filename + size, validate max 5MB
5. Add X-Request-ID header reading — generate UUID if not present, include in response header
6. Set a session_token cookie on /login — httponly=True, max_age=3600
7. Write middleware that logs: method, path, status_code, process_time for every request
8. Create a get_db() dependency with yield — simulate connection open/close with print statements
9. Add custom 422 handler — return {success: false, errors: [...]} instead of FastAPI default
10. Structure the app with 3 routers: users, auth, files — each in separate files

## 💬 INTERVIEW QUESTIONS TO NAIL

→ What happens when you have two Depends() that both depend on the same sub-dependency? (Answer: FastAPI reuses the same instance — once per request)
→ How would you prevent a 422 error from leaking internal field names to the client?
→ What's the difference between BackgroundTasks and Celery? When would you choose each?
→ How does FastAPI's dependency injection differ from Flask's approach?
→ Why should you use response_model= instead of just returning a dict?

# DAY 2
FastAPI

## FastAPI — Advanced & Production Patterns

*The topics that separate mid-level from senior — auth, streaming, testing*

📖 **DOCS** Advanced Docs: https://fastapi.tiangolo.com/advanced/

## TOPICS TO COVER

### 01 JWT Authentication

*Every backend role interviews JWT — know it well enough to implement from memory*

→ OAuth2PasswordBearer — token extraction from Authorization header
→ JWT structure: header.payload.signature — what each part contains
→ python-jose or PyJWT — creating and verifying tokens
→ Access token + Refresh token pattern — why both exist
→ get_current_user dependency — inject user into any route
→ Token expiry, HTTPException 401 on invalid token
→ Password hashing with passlib (bcrypt)

### 02 Advanced Dependency Injection

*Senior-level question: how do you apply auth to an entire router without decorating each route?*

→ Class-based dependencies: __call__ method, __init__ for config
→ Dependencies as security layers — layering auth + role check
→ Global dependencies: app = FastAPI(dependencies=[Depends(verify_token)])
→ Router-level dependencies: router = APIRouter(dependencies=[...])
→ Overriding dependencies in tests: app.dependency_overrides[get_db] = mock_db

### 03 Custom Middleware (Advanced)

*Middleware is where cross-cutting concerns live — logging, auth, rate limiting*

→ BaseHTTPMiddleware — call_next pattern
→ Starlette's pure ASGI middleware for performance
→ Rate limiting middleware using Redis
→ Tenant isolation middleware — extract tenant from header/subdomain
→ Middleware for request ID injection and structured logging

### 04 Background Tasks

*Know EXACTLY when NOT to use BackgroundTasks (long tasks, retries needed → use Celery)*

→ BackgroundTasks.add_task() — runs after response is sent
→ Why it's different from Celery: same process, no queue, fire-and-forget
→ Dependency injection inside background tasks
→ Use cases: sending email after signup, audit logging, webhook calls
→ Limitations: dies if process restarts, no retry, no visibility

### 05 WebSockets

*Real-time features are common in AI apps — know the connection manager pattern*

→ WebSocket endpoint: async def ws(websocket: WebSocket)
→ accept(), receive_text(), send_text(), close()
→ Connection manager pattern for broadcasting to multiple clients
→ Error handling: WebSocketDisconnect exception
→ Auth over WebSocket — query param token (can't use headers in browser WS)
→ Ping/Pong heartbeat to keep connection alive

## 06 HTTP Streaming & SSE

*CRITICAL for AI Engineer roles — every LLM API streams — you must know this cold*

→ StreamingResponse with generator function (yield chunks)
→ Server-Sent Events (SSE) — text/event-stream content type
→ LLM streaming pattern: stream token by token from OpenAI/Anthropic
→ Async generators for non-blocking streaming
→ Client-side: EventSource API (how frontend consumes SSE)

## 07 Testing FastAPI

*Senior roles require testing knowledge — know how to override dependencies for tests*

→ TestClient (sync) — from starlette.testclient import TestClient
→ AsyncClient (async) — httpx.AsyncClient with ASGITransport
→ pytest fixtures for app, client, test DB session
→ dependency_overrides — swap DB, swap auth user
→ Testing file uploads — BytesIO
→ Testing WebSocket connections
→ Coverage with pytest-cov

## 08 OpenAPI & API Design

*API design questions are common in full-stack interviews*

→ FastAPI auto-generates OpenAPI spec — how to customize it
→ Tags, summary, description, response_description
→ Deprecated flag on endpoints
→ API versioning strategies: /v1/users vs header-based
→ Security schemes in OpenAPI docs

## 🛠️ PRACTICAL EXERCISES

11. Build complete JWT auth: /register, /login (returns token), /me (protected route) — use passlib + python-jose
12. Create a class-based dependency CommonQueryParams(skip, limit) and use across 3 routes
13. Add router-level auth dependency to /admin/* routes — all admin routes auto-protected
14. Build /chat/stream endpoint using StreamingResponse — yield 'data: {chunk}\n\n' format (SSE)
15. Build a WebSocket /ws/chat — connection manager broadcasts messages to all connected clients
16. Write a middleware that rate limits by IP: max 10 req/min using a dict (simulate Redis)
17. Write tests: test /me with valid token, test /me with invalid token (expect 401), override get_db
18. Build /background/send-email — returns 202 immediately, logs 'email sent' 2 sec later in background

## 💬 INTERVIEW QUESTIONS TO NAIL

→ Walk me through what happens when a JWT expires mid-request — how do you handle it?
→ Why can't you pass Authorization header in a browser WebSocket connection? What's the workaround?
→ How is SSE different from WebSockets? Which would you use for an LLM chat interface and why?
→ How do you test a route that requires a logged-in user without hitting the real auth service?
→ What's the difference between app-level, router-level, and route-level dependencies?

# DAY 3

SQLModel + PG

## SQLModel + PostgreSQL — Full Database Mastery

*ORM, async queries, migrations, and PostgreSQL-specific power features*

📖 **DOCS** SQLModel Docs: https://sqlmodel.tiangolo.com/ | Alembic: https://alembic.sqlalchemy.org/en/latest/

## TOPICS TO COVER

### 01 SQLModel Fundamentals

*SQLModel is FastAPI-native — know the pattern for separating DB model from API schema*

- → SQLModel = SQLAlchemy + Pydantic — how it unifies ORM + validation
- → table=True vs table=False (Pydantic-only) models
- → Field() with primary_key, foreign_key, index, nullable
- → Optional fields and default values
- → Separation: UserBase, UserCreate, UserRead, UserUpdate pattern
- → from_orm() and model_validate() for conversion

### 02 Alembic Migrations

*Every backend interview: how do you handle schema changes in production without downtime?*

- → alembic init — project structure
- → env.py setup — point to SQLModel metadata
- → alembic revision --autogenerate -m 'add users table'
- → alembic upgrade head / downgrade -1
- → How autogenerate works — what it can/can't detect
- → Migration for adding column with default value to existing table

### 03 Async Sessions + CRUD

*Async DB is the production standard for FastAPI — know the full CRUD lifecycle*

- → AsyncSession from sqlalchemy.ext.asyncio
- → create_async_engine with asyncpg driver
- → Session lifecycle: get_session() as a dependency
- → CRUD: session.add(), await session.commit(), await session.refresh()
- → SELECT: select(User).where(User.email == email), await session.exec()
- → Delete: await session.delete(user), commit

### 04 Relationships

*N+1 query problem is a classic interview question — know selectinload as the async solution*

- → One-to-Many: User → Posts (Relationship() + back_populates)
- → Many-to-Many: Users ↔ Groups (link table with two FKs)
- → Lazy vs eager loading — selectin loading for async
- → joinedload vs selectinload — N+1 problem and solution
- → Cascade delete configuration

## 05 PostgreSQL — JSONB

*AI apps heavily use JSONB — LLM outputs, agent state, tool results are often stored as JSON*

→ JSONB vs JSON column type — why JSONB (indexed, queryable)
→ Column(JSONB) in SQLAlchemy / Field with sa_column
→ Querying JSONB: .where(User.metadata['role'].astext == 'admin')
→ GIN index on JSONB for fast queries
→ Use case: storing flexible metadata, config, AI tool outputs

## 06 Indexes & Query Optimization

*Performance questions are common for senior backend roles — EXPLAIN ANALYZE is key*

→ B-tree index (default) — equality and range queries
→ GIN index — JSONB, full-text search, array columns
→ Partial index — WHERE condition (e.g., index only active users)
→ Composite index — column order matters
→ EXPLAIN ANALYZE — reading query plans, spotting seq scans
→ Covering index — index includes all needed columns (no heap fetch)

## 07 Full-Text Search (FTS)

*AI Engineer question: when would you use PostgreSQL FTS vs a vector DB for search?*

→ tsvector column + GIN index for FTS
→ to_tsvector('english', content) vs stored computed column
→ to_tsquery, plainto_tsquery — user query processing
→ ts_rank for relevance scoring
→ When to use FTS vs pg_trgm vs Elasticsearch

## 08 Connection Pooling & Production Config

*Production knowledge — pooling questions separate junior from senior*

→ SQLAlchemy pool_size, max_overflow, pool_timeout
→ PgBouncer — external connection pooler, why it's used with serverless
→ Detecting connection leaks
→ Row-level security (RLS) — enable at Postgres, enforce in queries
→ PostgreSQL LISTEN/NOTIFY for real-time events without polling

## 🛠️ PRACTICAL EXERCISES

19. Define User, Post models with SQLModel — User has many Posts. Run Alembic migration.
20. Build full CRUD API for Users using AsyncSession — GET, POST, PUT, DELETE
21. Add pagination: GET /users?skip=0&limit=10 with total count in response
22. Add a JSONB column 'metadata' to User — store preferences. Query users by metadata field.
23. Create GIN index on the JSONB column, run EXPLAIN ANALYZE before/after
24. Implement FTS: add tsvector column on Post.content, build /search?q=... endpoint
25. Write a script that creates 10,000 users and benchmark query with/without index
26. Configure async engine with pool_size=10, max_overflow=20 — simulate connection reuse

## 💬 INTERVIEW QUESTIONS TO NAIL

→ What is the N+1 query problem? Show how selectinload solves it in async SQLAlchemy.

→ When would you choose JSONB over a normalized relational schema?
→ How does Alembic autogenerate work? What does it miss that you have to write manually?
→ Explain connection pooling — why does a pool of 10 connections serve hundreds of users?
→ How would you add a non-nullable column to a production table with millions of rows?

# DAY 4
Redis

## Redis — In-Memory Power for Production Systems

*Caching, Pub/Sub, rate limiting, locks — the backbone of fast backends*

📖 DOCS | Redis Docs: https://redis.io/docs/latest/ | redis-py: https://redis-py.readthedocs.io/

## TOPICS TO COVER

### 01 Data Structures — The Foundation

*Interviewers often give a scenario and ask 'which data structure would you use?'*

→ Strings: SET/GET/INCR/DECR/SETNX — most used, atomic ops
→ Hashes: HSET/HGET/HGETALL/HDEL — store objects (user sessions)
→ Lists: LPUSH/RPUSH/LPOP/RPOP/LRANGE — queues, activity feeds
→ Sets: SADD/SMEMBERS/SISMEMBER/SINTER — unique items, tags
→ Sorted Sets: ZADD/ZRANK/ZRANGE — leaderboards, priority queues
→ Why each structure exists — don't just memorize, understand use case

### 02 TTL, Expiry & Eviction

*Memory management is critical — know what happens when Redis runs out of memory*

→ EXPIRE key seconds / EXPIREAT key timestamp
→ PERSIST — remove TTL from a key
→ TTL / PTTL — check remaining time
→ Eviction policies: noeviction, allkeys-lru, volatile-lru, allkeys-lfu
→ When to use which eviction policy — cache vs session store

### 03 Caching Patterns

*Cache-aside is the most common pattern — know the tradeoffs between all three*

→ Cache-aside (lazy loading): check cache → miss → DB → set cache
→ Write-through: write to cache AND DB simultaneously
→ Write-behind (write-back): write cache first, async sync to DB
→ Cache invalidation strategies: TTL-based vs event-based
→ Cache stampede problem + solution (mutex lock / probabilistic early expiry)
→ FastAPI cache decorator pattern using Redis

### 04 Pub/Sub

*Pub/Sub appears in system design interviews — know when to use Streams instead*

→ PUBLISH channel message / SUBSCRIBE channel
→ Pattern subscribe: PSUBSCRIBE — wildcard channel matching
→ Use cases: real-time notifications, chat, event broadcasting
→ Limitation: no message persistence — if subscriber is down, messages lost
→ Redis Streams vs Pub/Sub — Streams have persistence and consumer groups

### 05 Rate Limiting Implementation

*Rate limiting is a staple system design + coding question — implement it from scratch*

→ Fixed window counter: INCR + EXPIRE per time window
→ Sliding window log: Sorted set with timestamp as score
→ Token bucket: more complex but smoother rate limiting
→ Per-user vs per-IP vs per-endpoint rate limiting
→ FastAPI middleware that checks Redis before request processing

## 06 Distributed Locks

*Distributed locks come up in system design — the NX EX pattern is the key insight*

→ SET key value NX EX seconds — atomic lock acquisition
→ Why SETNX alone is dangerous (no expiry → deadlock)
→ Redlock algorithm — acquiring lock across multiple Redis nodes
→ Lock renewal (expiry extension during long tasks)
→ Use case: prevent duplicate Celery task execution

## 07 Redis with FastAPI (Async)

*Practical integration — every production FastAPI app uses Redis for sessions/caching*

→ redis.asyncio — async client for FastAPI
→ Connection pool setup in lifespan
→ Dependency: get_redis() → yield client
→ Storing Pydantic models: json.dumps(model.dict())
→ Session storage pattern: hash per user session
→ Token blacklisting: add JWT jti to Redis on logout

### 🛠️ PRACTICAL EXERCISES

27. Implement cache-aside for GET /users/{id} — cache hit returns from Redis, miss fetches DB
28. Build rate limiter: 10 requests per minute per IP using fixed window (INCR + EXPIRE)
29. Implement sliding window rate limiter using Sorted Set — compare with fixed window
30. Build a leaderboard endpoint using Sorted Set — ZADD scores, ZREVRANGE top 10
31. Implement distributed lock: task starts only if no other instance holds the lock (SET NX EX)
32. Build Pub/Sub: /publish sends message to channel, /subscribe returns SSE stream from Redis
33. Token blacklist: on /logout, add token jti to Redis set with TTL = token remaining life
34. Session store: on login, store user data as Hash in Redis, read on each request

### 💬 INTERVIEW QUESTIONS TO NAIL

→ What's the difference between Redis Strings and Hashes for storing a user object?
→ Explain the cache stampede problem. How do you solve it with a Redis lock?
→ Why would you use Redis Streams instead of Pub/Sub for an order processing system?
→ How do you implement token invalidation (logout) with JWT? Walk me through the Redis approach.
→ Design a rate limiter that allows 100 requests per minute per user — which Redis structure and why?

# DAY 5
Celery

## Celery — Distributed Task Queues at Scale

*Async task execution, scheduling, retries, and FastAPI integration*

📖 DOCS | Celery Docs: https://docs.celeryq.dev/en/stable/

## TOPICS TO COVER

### 01 Celery Architecture

*Understanding the architecture is half the interview — draw it out mentally*

→ Components: Producer (FastAPI) → Broker (Redis) → Worker → Result Backend
→ Why Celery: separate long tasks from request-response cycle
→ Worker concurrency: prefork (CPU), eventlet/gevent (IO), solo
→ Task states: PENDING → STARTED → SUCCESS / FAILURE / RETRY
→ Concurrency model — prefork workers vs async workers

### 02 Task Definition & Execution

*Know the difference between delay() and apply_async() — async gives you control*

→ @app.task() — basic task definition
→ @app.task(bind=True) — access self for retry, request.id
→ task.delay() vs task.apply_async(countdown=, eta=, queue=)
→ AsyncResult — check task status and result from FastAPI
→ Task naming: auto vs manual — why explicit names matter in production
→ Soft time limit vs hard time limit

### 03 Retry Logic & Error Handling

*Retry with exponential backoff is a production must-know — interviewers love this*

→ self.retry(exc=e, countdown=5, max_retries=3)
→ autoretry_for — declarative retry on specific exceptions
→ Exponential backoff: countdown=2 ** self.request.retries
→ Dead letter queue — what happens after max retries
→ on_failure callback, on_retry callback
→ Idempotency — ensure retried tasks don't duplicate side effects

### 04 Canvas — Chain, Group, Chord

*Canvas patterns map directly to AI agent workflows — chain = sequential agent steps*

→ chain(task1.s(), task2.s()) — sequential pipeline
→ group(task1.s(), task2.s()) — parallel execution
→ chord(group, callback) — parallel then callback when all done
→ Signature .s() vs .si() — immutable signature (ignore previous result)
→ Use case: process 100 items in parallel (group), then aggregate (chord)

### 05 Celery Beat — Scheduled Tasks

*Beat is the cron replacement in modern Python — every production system needs it*

→ celery beat — the scheduler process
→ CELERY_BEAT_SCHEDULE — dict of periodic tasks
→ crontab() for cron-like scheduling
→ Dynamic scheduling with django-celery-beat or custom Beat
→ Use cases: cleanup jobs, report generation, cache warming

## 06 Monitoring & Production

*Observability is a senior-level concern — know how to debug stuck tasks in production*

→ Flower — real-time web UI for monitoring tasks
→ celery inspect active / reserved / scheduled
→ Task routing to specific queues — separate slow and fast tasks
→ Priority queues
→ Worker autoscaling: –autoscale=10,3
→ Detecting and handling task timeouts

## 07 FastAPI + Celery Integration

*Integration patterns are tested in full-stack + backend roles*

→ Shared celery app instance (celery_app.py module)
→ Sending task from FastAPI route — task.delay(), return task_id
→ GET /tasks/{task_id} — poll AsyncResult for status
→ Async result with WebSocket for real-time updates
→ Database session inside Celery task — create new session (not shared with FastAPI)
→ Config via environment variables — same config as FastAPI

### 🛠️ PRACTICAL EXERCISES

35. Create send_welcome_email task — delay 2s using countdown, return 'email sent'
36. Add retry with exponential backoff on simulated network failure (raise Exception randomly)
37. Build chain: validate_user.s() → create_account.s() → send_email.s() — sequential pipeline
38. Build group+chord: process_item.s() for 10 items in parallel → summarize.s() on results
39. Add beat schedule: cleanup_expired_sessions task every hour using crontab(minute=0)
40. FastAPI integration: POST /jobs returns {task_id}, GET /jobs/{task_id} returns {status, result}
41. Route heavy AI tasks to 'ai_queue', light tasks to 'default' queue — separate workers per queue
42. Start Flower, send 5 tasks, inspect them in the UI — understand states

### 💬 INTERVIEW QUESTIONS TO NAIL

→ What's the difference between a Celery worker and Celery beat? Do they run on the same process?
→ Your Celery task failed 3 times. How do you implement retry with exponential backoff?
→ How do you ensure a Celery task is idempotent? Why does idempotency matter with retries?
→ Walk me through how you'd architect: user uploads a large file → process it → notify when done
→ What's the difference between chain() and group() in Celery Canvas?

# DAY 6

LangChain + LangGraph

## LangChain + LangGraph — AI Engineer Depth

*LCEL, RAG, Tools, Agents, and Stateful Multi-Agent Workflows*

📖 DOCS

LangChain Docs: https://python.langchain.com/docs/ | LangGraph: https://langchain-ai.github.io/langgraph/

## TOPICS TO COVER

### 01 LCEL — LangChain Expression Language

*LCEL is the core abstraction — think of it as Unix pipes for LLM operations*

- → The pipe operator | — composing runnables
- → RunnablePassthrough, RunnableParallel, RunnableLambda
- → invoke(), stream(), batch(), ainvoke() — sync vs async
- → Input/output types flowing through the chain
- → When to use LCEL vs plain function calls
- → Fallbacks: chain.with_fallbacks([backup_chain])

### 02 Prompt Engineering in Code

*AI Engineer must know how to build production-grade prompts programmatically*

- → ChatPromptTemplate.from_messages() — system + human
- → MessagesPlaceholder for dynamic chat history
- → FewShotChatMessagePromptTemplate — in-context examples
- → PromptTemplate vs ChatPromptTemplate — when to use each
- → Partial prompts — pre-fill some variables
- → Prompt versioning strategy for production

### 03 Output Parsers & Structured Output

*Structured output is essential for tool-use and agent systems*

- → StrOutputParser — plain text
- → JsonOutputParser — stream JSON
- → PydanticOutputParser — validate LLM output against schema
- → with_structured_output() — most reliable for JSON extraction
- → Retry logic when LLM returns malformed output
- → Streaming partial JSON with JsonOutputParser

### 04 RAG Pipeline

*RAG is the #1 AI Engineer interview topic — know every step of the pipeline*

- → Document loaders: PDF, HTML, plain text
- → Text splitters: RecursiveCharacterTextSplitter — chunk_size, overlap
- → Embeddings: OpenAIEmbeddings, HuggingFaceEmbeddings
- → Vector stores: Chroma, FAISS, Qdrant, pgvector
- → Retriever patterns: similarity search, MMR (diversity)
- → RAG chain: retriever | format_docs | prompt | llm | parser

→ Hybrid search: semantic + keyword (BM25)

## 05 Tools & Tool Calling

*Tool calling is how LLMs interact with the real world — core AI Engineer skill*

→ @tool decorator — define tools with docstrings
→ StructuredTool — Pydantic schema for tool args
→ bind_tools() — attach tools to LLM
→ Tool execution flow: LLM decides → tool runs → result fed back
→ Tool error handling — what happens when tool fails
→ MCP (Model Context Protocol) — tool discovery over HTTP/stdio

## 06 LangGraph — StateGraph Fundamentals

*LangGraph is the production standard for stateful agents — know it architecturally*

→ StateGraph — graph with typed state (TypedDict or dataclass)
→ Nodes — Python functions that take state, return state update
→ Edges — static and conditional (routing logic)
→ START and END — entry and exit points
→ State reducers — how list state gets merged (operator.add)
→ Compile and run: graph.compile().invoke({...})

## 07 HITL, Memory & Advanced Patterns

*Your ESCAPE project already uses interrupt() — articulate this architecture clearly*

→ interrupt() — pause graph for human approval/input
→ Checkpointing: MemorySaver, SqliteSaver, RedisSaver
→ Thread-level persistence — resume from any checkpoint
→ Subgraphs — compose complex agents from smaller graphs
→ Multi-agent: Supervisor pattern, agent handoff
→ Streaming: .stream() vs .astream() vs .astream_events()

## 🛠️ PRACTICAL EXERCISES

43. Build a RAG pipeline: load PDF → split → embed → Chroma → retrieval chain with streaming
44. Create 3 tools: search_web, get_weather, calculate — bind to LLM, test tool selection
45. Build a ReAct agent with tools — trace the Thought/Action/Observation loop
46. Build StateGraph with 3 nodes: research → write → review — conditional edge to revise or finish
47. Add interrupt() before 'publish' node — simulate human approval in terminal
48. Add MemorySaver checkpointer — resume a paused graph from thread_id
49. Build supervisor agent: routes task to 'researcher' or 'coder' sub-agent based on query
50. Stream LangGraph execution: use .astream_events() and print each event type

## 💬 INTERVIEW QUESTIONS TO NAIL

→ Explain RAG — what problem does it solve, and what are the failure modes?
→ What's the difference between an Agent and a Chain in LangChain?
→ How does LangGraph's state machine differ from LangChain's AgentExecutor?
→ How do you implement human-in-the-loop in an AI workflow? Why is it important?

→ Your RAG is returning irrelevant chunks — how do you debug and fix it?

# DAY 7
Docker

## Docker — Containerization from Dev to Production

*Dockerfile best practices, Compose orchestration, networking, and production config*

📖 DOCS | Docker Docs: https://docs.docker.com/ | Compose: https://docs.docker.com/compose/

## TOPICS TO COVER

### 01 Dockerfile Fundamentals

*Layer caching is the #1 interview topic — explain why order matters*

→ Layer caching — order instructions from least to most frequently changed
→ COPY requirements.txt first, then pip install, then COPY . — cache deps
→ RUN vs CMD vs ENTRYPOINT — when to use each
→ .dockerignore — exclude __pycache__, .git, .env
→ ARG vs ENV — build-time vs runtime variables
→ USER instruction — never run as root in production

### 02 Multi-Stage Builds

*Multi-stage is expected knowledge for any senior backend/full-stack role*

→ FROM python:3.11 AS builder → FROM python:3.11-slim AS runtime
→ Why: smaller final image, compile in builder, copy artifacts to runtime
→ COPY --from=builder /app/dist /app
→ Use case: Next.js (build stage → serve static), Python (compile wheels → run)
→ Image size comparison: full vs slim vs alpine — tradeoffs

### 03 Docker Compose

*Compose is the daily tool — know the full-stack setup: FastAPI + Postgres + Redis*

→ services, networks, volumes — the three core concepts
→ depends_on with condition: service_healthy — wait for DB to be ready
→ Environment variables: env_file: .env vs environment:
→ Volume mounts for dev: hot reload with mounted source code
→ Named volumes vs bind mounts — persistence behavior
→ docker compose up --build, logs, exec, down -v

### 04 Networking

*Network DNS resolution is commonly misunderstood — services use service names, not localhost*

→ Bridge network — default, containers communicate by service name
→ Host network — container uses host's network stack
→ Services find each other by DNS: http://redis:6379, http://postgres:5432
→ Exposing ports: ports (host:container) vs expose (internal only)
→ Connecting external tools (e.g. DBeaver) to containerized Postgres

### 05 Health Checks & Production Config

*Health checks + restart policies are what make containers production-grade*

→ HEALTHCHECK instruction in Dockerfile
→ healthcheck in Compose: test, interval, retries, start_period
→ Restart policies: always, on-failure, unless-stopped
→ Resource limits: mem_limit, cpus
→ Secrets management: Docker secrets vs env vars
→ Logging drivers: json-file, fluentd

## 06 Production Patterns

*Interviewers want to know: how does your local compose translate to production?*

→ Full stack compose: FastAPI + PostgreSQL + Redis + Celery + Nginx
→ Nginx as reverse proxy — upstream to FastAPI, serve static files
→ Environment-specific compose files: docker-compose.yml + docker-compose.prod.yml
→ CI/CD: build → push to registry → deploy
→ Read-only container filesystem for security

### 🛠️ PRACTICAL EXERCISES

51. Write optimized Dockerfile for FastAPI app — multi-stage, non-root user, .dockerignore
52. Build and compare image sizes: python:3.11 vs python:3.11-slim — run du on site-packages
53. Write full docker-compose.yml: FastAPI + PostgreSQL + Redis + Celery worker + Flower UI
54. Add healthcheck to postgres service — depends_on: postgres: condition: service_healthy
55. Add Nginx service — proxy /api/* to FastAPI, serve / as static files
56. Test hot reload: mount source dir as volume, change code — FastAPI reloads without rebuild
57. Override compose for prod: docker-compose -f docker-compose.yml -f docker-compose.prod.yml up
58. Inspect a running container: docker exec -it <id> bash, check env vars, check processes

### 💬 INTERVIEW QUESTIONS TO NAIL

→ Why does Docker layer caching break when you COPY . before pip install? How do you fix it?
→ How do two containers in the same Compose network find each other? What's the DNS mechanism?
→ What's the difference between CMD and ENTRYPOINT? When would you use both together?
→ How do you handle database migrations in a containerized environment on deployment?
→ Your FastAPI container starts before PostgreSQL is ready — how do you handle this in Compose?

# DAY 8
Next.js

## Next.js — Modern Full-Stack with App Router

*Server components, data fetching patterns, API routes, and auth integration*

📖 DOCS | Next.js Docs: https://nextjs.org/docs (App Router section primarily)

## TOPICS TO COVER

### 01 App Router Architecture

*App Router is now the standard — know the file conventions cold*

→ app/ directory — file-based routing
→ page.tsx, layout.tsx, loading.tsx, error.tsx, not-found.tsx
→ Nested layouts — shared UI without re-rendering
→ Route groups: (auth)/, (dashboard)/ — organize without affecting URL
→ Dynamic routes: [id]/, [...slug]/, [[...slug]]/
→ Parallel routes and intercepting routes — advanced patterns

### 02 Server vs Client Components

*This is the #1 Next.js interview topic — know the rendering model deeply*

→ 'use client' directive — when and why
→ Server components: run on server, can access DB directly, no useState/hooks
→ Client components: interactivity, event handlers, browser APIs
→ Composition pattern: server wraps client — pass data as props
→ When NOT to make something a client component (avoid overuse)
→ Server Actions — async functions that run on server, called from client

### 03 Data Fetching

*Understanding when data runs server vs client is core to Next.js performance*

→ fetch() in server components — automatic request deduplication
→ cache: 'no-store' vs revalidate: 60 — dynamic vs ISR
→ generateStaticParams() for static route generation
→ Server Actions for mutations (POST/PUT/DELETE without API routes)
→ Client-side fetching with SWR or React Query
→ Streaming with Suspense + loading.tsx

### 04 API Routes

*Know when to use API routes vs calling FastAPI directly from the client*

→ app/api/route.ts — GET, POST, PUT, DELETE handlers
→ NextRequest and NextResponse
→ Route handlers vs Server Actions — when to use each
→ Middleware (middleware.ts) — runs before every request
→ CORS headers in API routes
→ Proxying requests to FastAPI backend from Next.js API routes

## 05 Authentication & State

*Full-stack role: know the end-to-end auth flow from Next.js to FastAPI*

→ NextAuth.js / Auth.js — providers, session handling
→ JWT session vs database session
→ Protecting routes with middleware.ts — check session, redirect
→ Passing auth token to FastAPI backend (Authorization header)
→ Environment variables: NEXT_PUBLIC_ vs server-only
→ Basic Zustand or React Context for global client state

## 06 Performance & Deployment

*For backend-heavy roles, just know enough Next.js to not look unfamiliar*

→ Image optimization: next/image — lazy loading, formats
→ Font optimization: next/font
→ Bundle analysis: @next/bundle-analyzer
→ Static vs dynamic rendering — when Next.js pre-renders
→ Deploying to Vercel vs self-hosting (Docker)
→ Environment variables in build vs runtime

## 🛠️ PRACTICAL EXERCISES

59. Build a dashboard layout: app/(dashboard)/layout.tsx wraps all dashboard pages
60. Create Server Component that fetches /api/users from FastAPI, renders list — no 'use client'
61. Create Client Component: search input that filters users — useState, fetch on change
62. Build API proxy route: app/api/users/route.ts — forward to FastAPI, add auth header
63. Add middleware.ts: redirect /dashboard to /login if no session cookie
64. Implement Server Action: form submission creates user — no separate API route needed
65. Add Suspense + loading.tsx to a slow data-fetching page — test loading state
66. Setup NextAuth with credentials provider — login calls FastAPI, stores JWT in session

## 💬 INTERVIEW QUESTIONS TO NAIL

→ What's the difference between a Server Component and a Client Component? How do you choose?
→ When would you use an API Route vs a Server Action?
→ How does Next.js handle data fetching differently from create-react-app?
→ How would you pass an auth token from Next.js to a FastAPI backend?
→ What is ISR (Incremental Static Regeneration)? When would you use it?

**DSA**
**Daily**

# DSA — Daily 2-Hour Structured Practice

*Topic-by-topic progression aligned with your 8-day prep window*

*Spend 1.5 hrs solving + 30 min reviewing solutions/patterns. Do NOT skip the review — that's where you internalize the pattern.*

| Day | Topic | Core Problems (LeetCode) | Pattern to Master | Difficulty |
|---|---|---|---|---|
| 1 | **Arrays & Hashing** | Two Sum, Valid Anagram, Group Anagrams, Product of Array Except Self | **HashMap for O(1) lookups** | Easy → Medium |
| 2 | **Two Pointers** | Valid Palindrome, 3Sum, Container With Most Water, Trapping Rain Water | **Left/right convergence** | Medium |
| 3 | **Sliding Window** | Max Profit (Buy/Sell), Longest Substring No Repeat, Min Window Substring | **Expand + shrink window** | Medium → Hard |
| 4 | **Stack & Queue** | Valid Parentheses, Min Stack, Daily Temperatures, Largest Rectangle Histogram | **Monotonic stack pattern** | Medium |
| 5 | **Binary Search** | Search Rotated Array, Find Minimum in Rotated, Koko Eating Bananas | **Left/right boundary conditions** | Medium |
| 6 | **Trees (BFS/DFS)** | Invert Binary Tree, Max Depth, Level Order Traversal, LCA, Serialize Tree | **Recursive DFS + iterative BFS** | Easy → Hard |
| 7 | **Graphs** | Number of Islands, Clone Graph, Course Schedule, Pacific Atlantic | **DFS/BFS + visited set** | Medium |
| 8 | **Dynamic Programming** | Climbing Stairs, Coin Change, Longest Common Subsequence, House Robber | **DP table — identify subproblem** | Medium → Hard |

**📌 DSA Strategy**

Solve the problem yourself first (25 min max). If stuck, look at the PATTERN only — not the full solution. After solving, ask: what's the time complexity? Can I reduce space? Could I solve this with a different data structure?

**⚠ Common Mistakes**

Do NOT grind 50 easy problems. 3 medium problems per day with deep understanding beats 10 easy problems you don't internalize. Quality >> Quantity.

| SD<br>Daily 45 min | System Design — Integrated Daily Practice<br>*Core concepts mapped to the stack you study each day* |
|---|---|

*Study 1 system design concept each day — aligned with the stack of that day. At end of session, try to design a mini-system using what you learned.*

| Day | SD Topic | Core Concepts | Design Question |
|---|---|---|---|
| 1-2 | **API Design** | REST vs GraphQL, versioning, pagination (cursor vs offset), idempotency keys | *Design the API for a Twitter-like feed* |
| 3 | **Database Design** | Normalization, denormalization, sharding, replication (leader-follower), read replicas | *Design the DB schema for an e-commerce platform* |
| 4 | **Caching Strategy** | Cache levels (L1/L2/CDN), cache invalidation, write-through vs cache-aside, hotspot keys | *Design a caching layer for a news feed* |
| 5 | **Message Queues** | Kafka vs RabbitMQ vs SQS, at-least-once vs exactly-once, consumer groups, DLQ | *Design an order processing system with async tasks* |
| 6 | **AI System Design** | Embedding pipeline, vector DB at scale, RAG latency, model serving, A/B testing LLMs | *Design a RAG-based customer support system* |
| 7 | **Infrastructure** | Load balancing (L4 vs L7), horizontal vs vertical scaling, health checks, blue-green deploy | *Design the infrastructure for a high-traffic API (10k RPS)* |
| 8 | **Full System** | CAP theorem, eventual consistency, rate limiting, CDN, monitoring (metrics, logs, traces) | *Design a URL shortener like bit.ly end-to-end* |

## ● SYSTEM DESIGN FRAMEWORK (Use in Every Interview)

| Step | What to Do |
|---|---|
| 1 | Clarify Requirements: Functional (what it does) + Non-functional (scale, latency, consistency) |
| 2 | Estimate Scale: DAU, QPS (reads vs writes), data size per record × total records = storage |
| 3 | High-Level Design: Draw the boxes — client, load balancer, API servers, DB, cache, queue |
| 4 | Deep Dive: Pick 2-3 components and go deep — DB schema, caching strategy, API design |
| 5 | Identify Bottlenecks: Single points of failure, hot partitions, slow queries — and solve them |
| 6 | Operational: Monitoring (metrics, alerts), logging, deployment strategy, disaster recovery |

# QUICK REFERENCE — OFFICIAL DOCS

| Stack | URL | Most Important Section |
|---|---|---|
| **FastAPI** | https://fastapi.tiangolo.com | *Tutorial → Advanced → Security* |
| **SQLModel** | https://sqlmodel.tiangolo.com | *Tutorial → Relationships → Async* |
| **Alembic** | https://alembic.sqlalchemy.org | *Tutorial → Auto-generate → Operations* |
| **Redis** | https://redis.io/docs/latest | *Data Types → Commands → Patterns* |
| **redis-py** | https://redis-py.readthedocs.io | *Commands → Connection Pools → Pub/Sub* |
| **Celery** | https://docs.celeryq.dev/en/stable | *Getting Started → Canvas → Periodic Tasks* |
| **LangChain** | https://python.langchain.com/docs | *Expression Language → RAG → Tools* |
| **LangGraph** | https://langchain-ai.github.io/langgraph | *Concepts → How-to → HITL* |
| **Docker** | https://docs.docker.com | *Dockerfile reference → Compose → Networking* |
| **Next.js** | https://nextjs.org/docs | *App Router → Data Fetching → Server Actions* |

## ● ROLE-SPECIFIC FOCUS AREAS

| AI Engineer | Backend Engineer | Full-Stack Developer |
|---|---|---|
| → LangGraph StateGraph + HITL<br>→ RAG pipeline end-to-end<br>→ Tool calling + MCP<br>→ SSE streaming from LLM<br>→ Vector DB + embeddings<br>→ Multi-agent orchestration<br>→ Redis for LLM caching<br>→ Celery for async AI tasks | → FastAPI DI + middleware depth<br>→ JWT auth implementation<br>→ Async SQLModel + Alembic<br>→ PostgreSQL indexing + FTS<br>→ Redis caching patterns<br>→ Celery + Beat + Canvas<br>→ Docker production setup<br>→ System design (DB + cache) | → Next.js App Router architecture<br>→ Server vs Client components<br>→ FastAPI ↔ Next.js auth flow<br>→ API routes + Server Actions<br>→ CORS configuration<br>→ Docker Compose full stack<br>→ Response models (no leaks)<br>→ SQL relationships + CRUD |

## READ → UNDERSTAND → BUILD

*Every single topic. No exceptions. Docs first, then practical. This is the only way.*